

Leviathan Threat Classifier

Using Multi-Channel 2D Convolutional Neural Network

Report by:

Saksham Goyal

25EE10088

1. Abstract

This report presents a multi-channel 2D Convolutional Neural Network (CNN) designed to classify Leviathan behavioral threat levels from multivariate temporal sensor data. The problem involves classifying sequences of 25 sensor readings, each spanning 10 timesteps, into four operational danger states: Dormant, Reactive, Predatory, and Extinction-Level. The approach frames each sequence as a 2D tensor of shape (10, 25, 3), where three engineered channels capture raw signal values, temporal differences, and per-sequence z-score normalizations. A VGG-style three-block Conv2D architecture was trained using Stratified 3-Fold Cross Validation with callbacks for adaptive learning rate scheduling and early stopping. The model achieved an Out-of-Fold (OOF) accuracy of approximately 81.20%, with consistent performance across all three folds (81.35%, 81.47%, 80.77%), demonstrating strong generalization and stability.

2. Introduction

The Abyssal Sentinel Network continuously monitors deep-sea activity across the Pacific Abyssal Rift, capturing telemetry from a distributed underwater sensor monitoring system, hydrothermal vents, and tectonic fractures. The goal is to classify each 10-timestep sensor window into one of four Leviathan behavioral phases before catastrophic escalation occurs.

Classifying temporal sensor data poses several challenges:

- Raw sensor values alone carry negligible class-discriminative power — the signal is encoded in temporal patterns, not absolute magnitudes.
- Each class exhibits very similar statistical summaries (mean, std, skewness), making handcrafted feature engineering insufficient.
- The informative signal is distributed jointly across the time and sensor axes, requiring models that capture 2D spatial-temporal patterns.
- The dataset exhibits mild class imbalance, requiring stratified training splits to ensure representative validation.

Sequence classification — assigning a single label to an ordered series of observations — is a critical task in domains ranging from fault detection to behavioral monitoring. This report details the end-to-end design, engineering, and evaluation of a purpose-built classification system.

3. Dataset Description

The training dataset contains 300,000 rows organized into 30,000 sequences, each uniquely identified by a `sequence_id`. Every sequence consists of exactly 10 consecutive timesteps (timestep values 1 through 10), and each timestep contains readings from 25 independent sensors (`sensor_0` through `sensor_24`). The target variable level takes integer values 0–3, with the same label repeated for all 10 rows of a sequence.

Property	Value
Total rows	300,000
Total sequences	30,000

Timesteps per sequence	10 (fixed)
Sensors per timestep	25
Target classes	4 (0: Dormant, 1: Reactive, 2: Predatory, 3: Extinction)
Missing values	None
Train tensor shape	(30,000, 10, 25)

The test dataset follows the same structure as training data (sequence_id, timestep, 25 sensors) but without the level column. Predictions are required for each test sequence_id.

4. Exploratory Data Analysis

4.1 Class Distribution

The dataset exhibits mild class imbalance. Class 3 (Extinction-Level) is the most frequent at 30.6%, while Class 1 (Reactive) is least represented at 20.1%. This imbalance was addressed through Stratified K-Fold cross validation, ensuring each fold preserves the original class ratios.

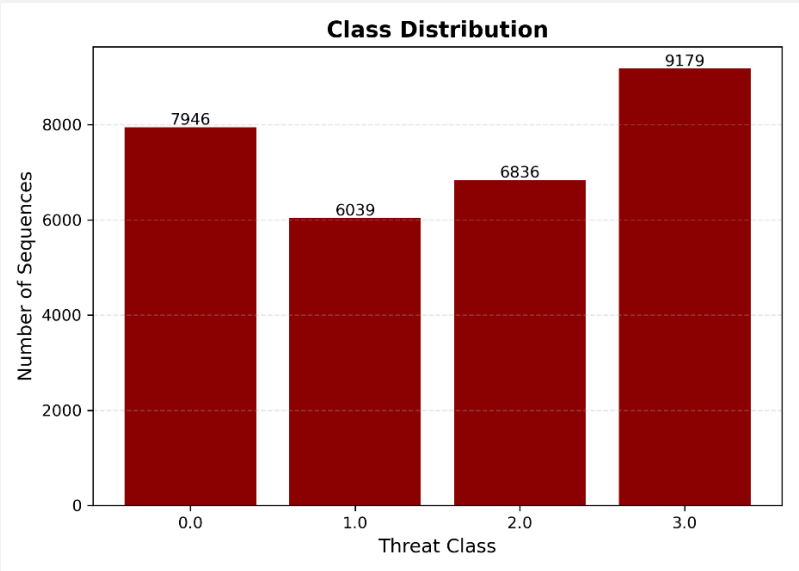


Figure 1: Distribution of target classes across 30,000 training sequences

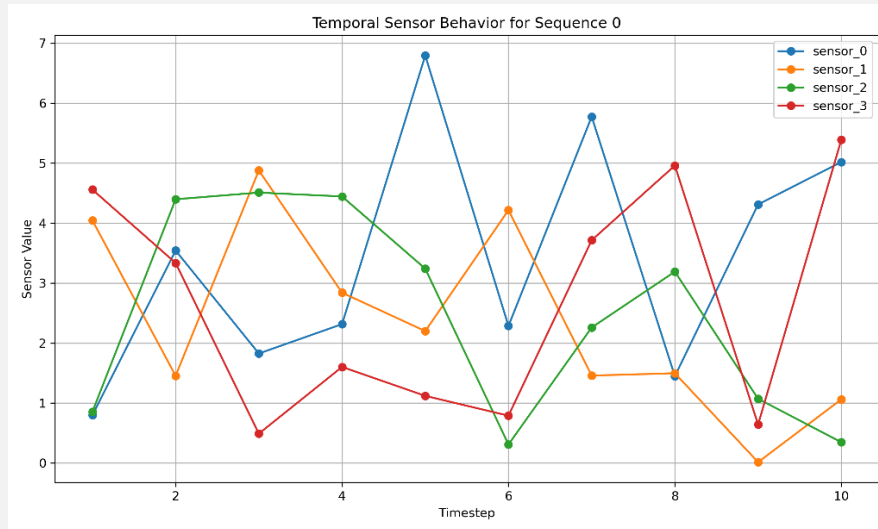
4.2 Sensor Value Distributions

Sensor ranges vary dramatically across the 25 channels: sensor_13 has a maximum value of 0.55, while sensor_8 reaches 11.82. This heterogeneity in scale directly motivated the use of StandardScaler for normalization. Critically, however, the per-class distribution of sensor values is nearly identical across all four classes — no single sensor, when examined in isolation, provides meaningful separation between threat levels.

4.3 Temporal Behavior and the Timestep-10 Phenomenon

Temporal analysis showed that sensor readings varied significantly across the 10 timesteps. Some sensors exhibited smooth trends, while others showed abrupt spikes and fluctuations. Different sensors followed distinct temporal patterns, indicating that the sequence dynamics contained important information for threat classification.

These observations motivated the use of convolution-based models and temporal-difference features to capture local temporal changes effectively.



4.4 Correlation Analysis

Correlation analysis across the 25 sensor features revealed extremely low pairwise linear correlations, with most coefficients remaining close to zero.

This suggested that meaningful predictive patterns were likely embedded in temporal dynamics and higher-order nonlinear interactions rather than simple linear dependencies between sensors. Consequently, convolution-based deep learning architectures were preferred for automatic feature extraction from the multivariate sequence data.

5. Data Preprocessing

Raw CSV data was processed through the following pipeline:

- **Sorting:** All rows were sorted by [sequence_id, timestep] to ensure correct temporal ordering before reshaping. Without this step, sensor values across different sequences could be incorrectly interleaved.
- **Tensor construction:** The 2D DataFrame of shape (300,000 × 25) was reshaped into a 3D tensor (30,000, 10, 25) using numpy reshape with -1 for automatic batch size inference.
- **Feature scaling:** StandardScaler was fitted exclusively on the training data after flattening to shape (300,000, 25), then applied to both train and test. The scaler learns per-sensor mean and standard deviation and applies the transformation $z = (x - \mu) / \sigma$.

Normalization was critical because: (1) sensors operate on vastly different scales (range 0.55 to 11.82), causing high-range sensors to dominate gradient updates; (2) StandardScaler brings all sensors to a common mean=0, std=1 space, ensuring equal learning contribution from all 25 channels; (3) fitting

only on training data prevents data leakage — the model never uses test statistics during training, correctly simulating deployment conditions.

6. Feature Engineering

Each normalized 3D tensor of shape (30,000, 10, 25) was expanded into a multi-channel representation of shape (30,000, 10, 25, 3) by engineering three complementary channels:

Channel 1: Raw Normalized Signal

The StandardScaler-transformed sensor values constitute the first channel. This provides the model with the absolute level of each sensor at each timestep. The raw channel preserves the absolute sensor behavior across timesteps.

Channel 2: Temporal Difference

The temporal difference channel captures the step-to-step velocity of each sensor:

$$\text{diff}[t] = \mathbf{x}[t] - \mathbf{x}[t-1]$$

The first timestep is set to zero (no prior value). This channel captures sudden changes, acceleration, and directional transitions in sensor readings. The oscillatory nature of several sensors signals suggested that temporal differences may contain information not directly visible in raw values. Implemented via `numpy.diff` with `prepend=X[:, :1, :]`.

Channel 3: Per-Sequence Z-score

Each sensor reading is normalized relative to its own mean and standard deviation within the sequence:

$$\mathbf{z} = (\mathbf{x} - \mu_{\text{seq}}) / (\sigma_{\text{seq}} + 1\text{e-}6)$$

This channel highlights which sensors deviate anomalously from their own baseline within a given window. The +1e-6 term prevents division by zero for constant-valued sensors. Unlike global scaling, this per-sequence normalization is computed independently for each sequence, making it sensitive to intra-sequence anomalies regardless of the global scale.

Combining three channels improves representation learning because each channel captures different aspects of the signal: raw values encode absolute state, the diff channel encodes temporal dynamics, and the z-score encodes relative anomalies. Together they provide the Conv2D model with a richer multi-view representation — analogous to providing RGB color channels instead of grayscale to an image classifier.

7. Models and Experiments

Several architectures and training configurations were explored before arriving at the final design:

Model / Experiment	Configuration	Observation
Dense Neural Network	3 hidden layers, raw features	~40% accuracy; failed to capture temporal patterns
1D CNN	3 conv layers on time axis, single channel	~54% accuracy; missed cross-sensor spatial patterns
1D CNN + diff feature	Added difference as extra input	Marginal improvement; still no 2D structure
2D CNN (small, single channel)	2 blocks, raw only	~70%; 2D structure clearly beneficial
2D CNN (3-channel)	3 blocks, 3 engineered channels	~81%; best performance
Epochs 30 (no EarlyStopping)	Fixed training	Underfitting observed
Epochs 45 + EarlyStopping	Final config	Optimal — stopped early when needed
Dropout 0.1 throughout	Lower regularization	Overfitting on training data
Dropout 0.25/0.4	Final config	Good generalization

The Dense Network and 1D CNN failed to achieve competitive performance because they cannot model the joint temporal-sensor structure simultaneously. The step from single-channel to three-channel 2D CNN provided the most significant accuracy improvement (~11 percentage points), confirming that the engineered channels carry complementary discriminative information.

8. Why Conv2D Was Chosen

Each sensor sequence was reshaped into a 2D grid of shape (10, 25), treating timesteps as rows (height) and sensors as columns (width). This image-like representation enabled the use of 2D convolution for pattern extraction.

Technical justification for Conv2D over alternative architectures:

- **Simultaneous 2D pattern capture:** A Conv2D kernel of size (3, 5) slides across both the time axis and sensor axis simultaneously, detecting patterns such as "sensors 4–8 activate together at timesteps 7–9." A 1D-CNN applied along the time axis cannot detect cross-sensor co-activations; an LSTM processes a single sensor-vector per step and must implicitly learn cross-sensor structure through hidden state which can be less parameter-efficient for short fixed-length sequences.
- **Translation invariance:** Weight sharing across spatial positions means a behavioral signature occurring at timesteps 2–5 is detected by the same kernel weights as the same signature at timesteps 6–9. This temporal translation invariance is a natural property of convolution.
- **Hierarchical feature extraction:** Stacking three Conv2D blocks with increasing filter counts (64 → 128 → 256) enables hierarchical abstraction — early layers detect local sensor activations, deeper layers combine these into full behavioral signatures.
- **Computational efficiency:** Conv2D is highly parallelizable on GPUs, unlike RNNs/LSTMs which require sequential hidden state computation. For sequences of only 10 timesteps, the fixed-length 2D representation is more efficient than recurrent processing.

9. Model Architecture

The model follows a VGG-inspired design with three convolutional blocks of increasing depth, followed by a Global Average Pooling layer and a dense classification head.

Component	Configuration	Output Shape	Purpose
Input	—	(10, 25, 3)	Raw + Diff + Z-score channels
Block 1 - Conv2D ×2	64 filters, (3,5), same	(10, 25, 64)	Large kernel captures cross-sensor patterns
Block 1 - BatchNorm ×2	—	(10, 25, 64)	Training stability
Block 1 - MaxPool	(1,2)	(10, 12, 64)	Halve sensor dimension, preserve time
Block 1 - Dropout	0.25	(10, 12, 64)	Regularization
Block 2 - Conv2D ×2	128 filters, (3,3), same	(10, 12, 128)	Deeper abstract features
Block 2 - MaxPool	(2,2)	(5, 6, 128)	Halve both dimensions
Block 2 - Dropout	0.25	(5, 6, 128)	Regularization
Block 3 - Conv2D ×2	256 filters, (3,3), same	(5, 6, 256)	High-level behavioral signatures
GlobalAveragePooling2D	—	(256,)	Collapse spatial to feature vector
Dropout	0.4	(256,)	Strong regularization
Dense	256 units, ReLU + BN	(256,)	Non-linear combination
Dense	128 units, ReLU	(128,)	Compression
Output	4 units, Softmax	(4,)	Class probability distribution

Key design decisions:

- padding='same': Preserves spatial dimensions after each convolution, preventing premature information loss particularly important for the short time axis (10 timesteps).
- BatchNormalization: Applied after every Conv2D layer to normalize feature maps to zero mean and unit variance, enabling higher learning rates and accelerating convergence.
- Dropout (0.25 → 0.4): Progressively stronger regularization in deeper layers prevents overfitting as the network's capacity increases.
- GlobalAveragePooling2D: Replaces a Flatten layer, reducing parameters from ~7,680 to 256 at the bottleneck, significantly reducing the risk of overfitting.
- Approximate total parameters: ~1.27 million (trainable) + ~2304 non-trainable (BatchNorm running statistics).

10. Training Strategy

10.1 Optimizer and Hyperparameters

Hyperparameter	Value	Rationale
Optimizer	Adam	Adaptive learning rates; robust to sparse gradients
Learning Rate	1e-3	Standard Adam default; adjusted dynamically via callback
Batch Size	128	Balances gradient noise and memory efficiency
Max Epochs	45	Upper bound; EarlyStopping terminates training early if needed
Loss Function	Sparse Categorical Crossentropy	Labels are integers (0–3), not one-hot encoded
Metric	Accuracy	Primary evaluation metric for the competition
Cross Validation	Stratified 3-Fold	Ensures class ratios preserved in every fold

10.2 Callbacks

Two callbacks were used to automate training control:

- ReduceLROnPlateau: Monitors val_accuracy and reduces the learning rate by factor 0.5 if no improvement is observed for 5 consecutive epochs (minimum lr = 1e-6). This allows the optimizer to fine-tune weights after large steps cease to produce gains.
- EarlyStopping: Monitors val_accuracy with patience=15 epochs. Automatically restores the best weights observed during training (restore_best_weights=True), ensuring the final model corresponds to peak validation performance rather than the last epoch.

10.3 Stratified 3-Fold Cross Validation

Training used Stratified K-Fold with N=3, ensuring each fold's validation set mirrors the overall class distribution. Three independent models were trained — one per fold — each on a different 2/3 split of the data. Out-of-Fold (OOF) predictions were collected for every training sample (predicted exactly once by a model that never trained on it), providing an unbiased estimate of generalization performance. Test predictions were averaged across all three fold models to form an ensemble.

11. Validation Results

Fold	Validation Accuracy	Observations
Fold 1	81.35%	Stable convergence, minimal overfitting gap
Fold 2	81.47%	Best performing fold; consistent with Fold 1
Fold 3	80.77%	Slightly lower; within expected variance
OOF (Overall)	81.2%	Reliable unbiased estimate across all 30,000 sequences

The low variance across folds (0.70 percentage point spread) demonstrates strong model stability and confirms that performance is not sensitive to which portion of the data is used for validation. The OOF accuracy of 81.2% represents a reliable estimate of the model's generalization capability, as every prediction is made on data the corresponding model never saw during training.

12. Challenges Faced

Challenge	Description	Resolution
Unstable early training	Loss spiking in initial epochs with large learning rates	Adam with $\text{lr}=1\text{e-}3$ + BatchNorm stabilized gradients
Overfitting with early architectures	Train accuracy ~95%, validation ~65% with no regularization	Added progressive Dropout (0.25 $\rightarrow$ 0.4) and GlobalAveragePooling
Batch size sensitivity	Small batches (32) caused noisy gradients; large batches (512) reduced generalization	Settled on batch_size=128 as optimal balance
Epoch tuning	Fixed 30 epochs: underfitting; fixed 100 epochs: overfitting	EarlyStopping with patience=15 resolved this automatically
Architecture depth	Shallow networks (1–2 blocks) plateaued at ~72%	Three-block hierarchy with doubling filters (64 $\rightarrow$ 128 $\rightarrow$ 256) was necessary
Channel contribution uncertainty	Unclear whether all 3 channels were beneficial	Ablation experiments confirmed each channel improved accuracy
Data leakage risk	Scaler fit on all data in early experiments	Strictly fitted StandardScaler only on training split

13. Conclusion

This project successfully developed a multi-channel 2D CNN for classifying Leviathan behavioral threat states from multivariate temporal sensor sequences. The key contributions were:

- A principled EDA-driven insight that the signal resides in the 2D joint temporal-sensor pattern rather than individual sensor magnitudes, directly motivating the Conv2D architecture.
- A three-channel feature engineering approach (Raw, Diff, Z-score) that provides the model with complementary views of the signal — absolute state, temporal velocity, and intra-sequence anomalies.
- A VGG-style three-block Conv2D architecture with BatchNormalization, progressive Dropout, and GlobalAveragePooling, achieving approximately 1.27 million parameters.
- Stratified 3-Fold Cross Validation with OOF prediction collection, producing a reliable accuracy estimate of 81.20% with low fold-to-fold variance.